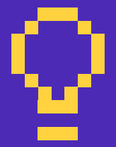
Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs



DAY 3

Security

Authentication, ownership, and permissions

Tuesday, October 27, 2026

Your five-day journey

Yesterday the API learned CRUD. Today every request gets an identity.

01

Foundation

Project, Task model, migrations, admin

DONE

02

Core API

CRUD endpoints and validation

DONE

03

Security

Tokens, ownership, isolation

TODAY

04

Quality

8 tests, settings from the environment

05

Publish

Deploy, verify, README, live demo

Today's session

Learn the security model, test it with two users, then apply it to the project

Explain + live demo
~60 min

Exercise
~60 min

Break
short

Project
~60 min

Exit check
end of day

Priority rule: finish the exercise core, then move to the project. Stretch work can wait.

Learning outcomes

By the end of today you can...

01 Secure API requests with DRF token authentication

02 Explain authentication versus authorization

03 Filter each user's list to their own records

04 Assign ownership on the server, not from the request body

05 Block cross-user retrieve, update, and delete

Authentication and authorization

Security starts with identity, then checks access

Authentication · who is calling?
A token proves the caller's identity

Authorization · what may they do?
Ownership limits the caller's data

401
no or invalid token

403
permission is denied

404
another user's object is invisible

In this program, another user's task returns 404, not 403.

Token authentication

A login request exchanges credentials for a key used on later requests

Username + password → **POST /api/auth/token/** → **Token key** → **Authorization: Token <key>**

Token first

List TokenAuthentication before SessionAuthentication so missing credentials return 401.

A token acts like a password. Never put it in Git, screenshots, or shared evidence.

Postman request setup

Keep the server address and token in an environment

Environment

`base_url = http://127.0.0.1:8000`

token = empty until login

Login request

POST `{{base_url}}/api/auth/token/`

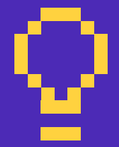
Save the returned token

Protected request

`Authorization: Token {{token}}`

On every API request

```
// post-response script on the login request
const data = pm.response.json();
pm.environment.set("token", data.token);
```



LIVE DEMO

Attack and fix

Two users: Bob reaches Alice's task, then we close the hole

Bob requests Alice's task
before the fix

Unfiltered queryset
returns it

Filter get_queryset()
by request.user

Bob gets 404
after the fix

perform_create already sets owner = request.user, so a client-supplied owner is ignored

Ownership is a query boundary

The safest object permission starts before the object is found

Alice's queryset
Alice's tasks only

Bob's queryset
Bob's tasks only

GET /api/tasks/{alice_id}/

Bob searches Bob's queryset

Alice's object is absent

404

404 hides that another user's data even exists.

Defense in depth

The core filter is required. An object-level check is optional reinforcement.

Core · P4
get_queryset() returns only the caller's objects

Server-owned field
owner is read-only; perform_create sets it (Day 2)

Stretch · P5
IsOwner.has_object_permission as a second layer

P5 and P6 are optional. Finish the two-user check first.

HOUR 2 · EXERCISE

Personal Reading List

Alice and Bob each get a token and see only their own items

Core	Postman	Stretch
about 45 min	included	if time remains

Run Tasks 1 and 2 until six tests pass

Exercise: what you build

Use Postman and the tests to prove the security boundary

01 Token authentication: settings, migrate, login URL

02 Ownership: filter the list and set the owner

03 Stretch: target date not in the past, no duplicate book per user

- Test list, create, retrieve, and update as Alice and as Bob
- Record results without copying tokens into notes or screenshots

BREAK

Short break

After the break: secure your own Task Management API

HOUR 3 · PROJECT

Secure the Task API

Token authentication and ownership for your project

Commit Day 2 → Copy checkpoint → P1–P4 → Two-user check → Commit + push

Copy project/checkpoints/day-03 after saving your work. Keep your Git history, .venv, and database.

Day 3 project increment

Four core TODOs, two stretch

TODO	What you build
P1	authtoken app in settings, then migrate
P2	authentication classes, Token before Session
P3	token endpoint at /api/auth/token/
P4	get_queryset filtered by request.user
P5 · stretch	IsOwner object permission
P6 · stretch	owner shown as username

Then run the two-user check. Save evidence without tokens.

The two-user security check

The API should protect the boundary in every direction

Step	Request	Expect
Login	valid credentials	200 + token
Wrong password	invalid credentials	400
No token	GET /api/tasks/	401
Own list	Alice lists tasks	200, Alice's only
Create	Alice sends Bob as owner	201, owner = Alice
Cross-user read	Bob GETs Alice's task	404
Cross-user edit	Bob PATCHes Alice's task	404, unchanged
Cross-user delete	Bob DELETEs Alice's task	404, still exists

Day 3 exit check

Ready for Day 4 when all six pass

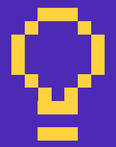
01	Valid credentials return a token; no token returns 401
02	A client-supplied owner is ignored
03	Each user's list holds only their own records
04	Cross-user retrieve, update, delete → 404, nothing leaks
05	The two-user check is recorded without tokens
06	The commit is pushed

Tomorrow these checks become automated tests

Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs



Day 3 complete

The API now knows who is calling and whose data they may see.

Next: automated tests, environment configuration, and deployment packages